

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине : «технологии и инструментальные средства проектирования
интеллектуальных систем»

По теме : «**Модели решения задач в интеллектуальных системах** »

Выполнил :
студент 4 курса
группы ИИ-25
Скачков Е.В
Проверила :
Андренко К.В.

Брест 2026

Цель работы: Реализовать **многослойный** персептрон (MLP) архитектуры **2-2-1** с **сигмоидной** функцией активации для решения задачи типа **«исключающее ИЛИ»** (XOR), в которой входные и выходные значения представлены не бинарными 0 и 1, а произвольными вещественными (целыми) числами из диапазона от -10 до 10 , соответствующими логическим классам 0 и 1. Провести сравнение эффективности обучения и качества решения задачи многослойным персептроном в сопоставлении с **однослойным персептроном**, показав принципиальную линейную неразделимость задачи XOR.

Вариант 9 ($c_0 = 3$, $c_1 = -8$)

A	B	$A \oplus B$
3	3	3
3	-8	-8
-8	3	-8
-8	-8	3

Ход работы:

```
import numpy as np
```

```
C0, C1 = 3, -8
```

```
DATA = [(3, 3, 3), (3, -8, -8), (-8, 3, -8), (-8, -8, 3)] # (A, B, A xor B)
```

```
#нормализация
```

```
T_LO, T_HI = 0.1, 0.9
```

```
Y_MIN, Y_MAX = min(C0, C1), max(C0, C1)
```

```
def norm_x(v): return v / 10.0
```

```
def norm_y(y): return T_LO + (y - Y_MIN) * (T_HI - T_LO) / (Y_MAX - Y_MIN)
```

```
def denorm_y(t): return Y_MIN + (t - T_LO) * (Y_MAX - Y_MIN) / (T_HI - T_LO)
```

```
X = np.array([[norm_x(a), norm_x(b)] for a, b, _ in DATA])
```

```
T = np.array([norm_y(y) for _, _, y in DATA])
```

```
Y_REAL = np.array([y for _, _, y in DATA], dtype=float)
```

```
sigmoid = lambda s: 1.0 / (1.0 + np.exp(-s))
```

```
def total_error(predict):
```

```
    return 0.5 * sum((T[i] - predict(X[i])) ** 2 for i in range(len(X)))
```

```
def accuracy(predict):
```

```
    ok = 0
```

```
    for i in range(len(X)):
```

```

y = denorm_y(predict(X[i]))
ok += abs(y - C0) < abs(y - C1) if Y_REAL[i] == C0 else abs(y - C1) < abs(y - C0)
return ok / len(X)

```

class MLP:

```

def __init__(self, rng):
    self.W1 = rng.uniform(-0.5, 0.5, (2, 2)); self.b1 = rng.uniform(-0.5, 0.5, 2)
    self.W2 = rng.uniform(-0.5, 0.5, 2); self.b2 = rng.uniform(-0.5, 0.5)

```

```

def forward(self, x):
    self.h = sigmoid(x @ self.W1 + self.b1)
    self.y = sigmoid(self.h @ self.W2 + self.b2)
    return self.y

```

```

def predict(self, x): return self.forward(x)

```

```

def train_step(self, x, t, lr):
    y = self.forward(x)
    d_out = (y - t) * y * (1 - y)
    d_hid = d_out * self.W2 * self.h * (1 - self.h)
    self.W2 -= lr * d_out * self.h; self.b2 -= lr * d_out
    self.W1 -= lr * np.outer(x, d_hid); self.b1 -= lr * d_hid

```

class SLP:

```

def __init__(self, rng):
    self.w = rng.uniform(-0.5, 0.5, 2); self.b = rng.uniform(-0.5, 0.5)

```

```

def predict(self, x): return sigmoid(x @ self.w + self.b)

```

```

def train_step(self, x, t, lr):
    y = self.predict(x)
    d = (y - t) * y * (1 - y)
    self.w -= lr * d * x; self.b -= lr * d

```

def train(net, lr, Ee, max_epochs, rng):

```

for epoch in range(1, max_epochs + 1):
    for i in rng.permutation(len(X)):
        net.train_step(X[i], T[i], lr)
    Es = total_error(net.predict)
    if Es <= Ee:
        return epoch, Es, True
return max_epochs, Es, False

```

def report(name, net, epochs, Es, ok):

```

print(f"\n=== {name} ===")
print(f"Критерий остановки достигнут: {'да' if ok else 'НЕТ'}; эпох: {epochs}; Es = {Es:.6f}")
print(f"{'A':>4} {'B':>4} {'цель':>5} {'выход':>8} класс")
for (a, b, y), x in zip(DATA, X):
    out = denorm_y(net.predict(x))
    cls = 'c0' if abs(out - C0) < abs(out - C1) else 'c1'
    print(f"{'a':>4} {'b':>4} {'y':>5} {'out':>8.3f} {'cls}'")
print(f"Accuracy: {accuracy(net.predict) * 100:.0f}%")

```

```

def run_mode(net):
    print(f"\nРежим функционирования (c0={C0}, c1={C1}). Пустая строка — выход.")
    while True:
        s = input("Введите А и В из [-10; 10] через пробел: ").strip()
        if not s: break
        try:
            a, b = map(float, s.split())
            assert -10 <= a <= 10 and -10 <= b <= 10
        except Exception:
            print("Некорректный ввод."); continue
        y = denorm_y(net.predict(np.array([norm_x(a), norm_x(b)])))
        cls = 'c0' if abs(y - C0) < abs(y - C1) else 'c1'
        print(f"ŷ = {y:.3f} -> ближе к {cls} ({C0 if cls == 'c0' else C1})")

if __name__ == "__main__":
    LR, EE, MAX_EP = 0.5, 0.001, 200_000

    for seed in range(100):
        rng = np.random.default_rng(seed)
        mlp = MLP(rng)
        ep, Es, ok = train(mlp, LR, EE, MAX_EP, rng)
        if ok: break
        print(f"seed={seed}: не сошлась (Es={Es:.4f}), перезапуск")
    print(f"\n(seed = {seed})")
    report("Многослойный персептрон 2-2-1", mlp, ep, Es, ok)

    slp = SLP(np.random.default_rng(0))
    ep2, Es2, ok2 = train(slp, LR, EE, 20_000, np.random.default_rng(0))
    report("Однослойный персептрон", slp, ep2, Es2, ok2)

    run_mode(mlp)

```

=== Многослойный персептрон 2-2-1 ===

Критерий остановки достигнут: да; эпох: 3925; $E_s = 0.000998$

A	B	цель	выход	класс
3	3	3	2.703	c0
3	-8	-8	-7.795	c1
-8	3	-8	-7.565	c1
-8	-8	3	2.759	c0

Accuracy: 100%

=== Однослойный персептрон ===

Критерий остановки достигнут: НЕТ; эпох: 20000; $E_s = 0.320014$

A	B	цель	выход	класс
3	3	3	-2.561	c1
3	-8	-8	-2.520	c1
-8	3	-8	-2.531	c1
-8	-8	3	-2.490	c0

Accuracy: 75%

Режим функционирования ($c_0=3$, $c_1=-8$). Пустая строка – выход.

Введите A и B из $[-10; 10]$ через пробел: -5 3

$\hat{y} = -5.843 \rightarrow$ ближе к $c_1 (-8)$

Вывод:

1. Однослойный персептрон не решил задачу. Он строит одну разделяющую прямую, а точки класса c_0 $((3,3)$ и $(-8,-8))$ и класса c_1 $((3,-8)$ и $(-8,3))$ лежат в противоположных углах квадрата. Никакая прямая их не разделяет. Лучшее, что он находит, это 3 из 4 примеров (75%).
2. MLP 2-2-1 решил задачу за 3925 эпох. Число эпох зависит от начальных весов. В коде при зависании в локальном минимуме предусмотрен перезапуск с другим seed.
3. Произвольные числа вместо 0/1 потребовали нормализации целей. Без неё сигмоида на выходе просто не смогла бы выдать -8 или 3 . Альтернатива — линейный выходной нейрон, но по условию сигмоида нужна на всех слоях.

4. Скрытый слой нужен, потому что два нейрона скрытого слоя задают две прямые, то есть преобразуют пространство. В нём классы становятся линейно разделимыми для выходного нейрона.
5. Сложность задачи не изменилась. Это тот же XOR: после линейного масштабирования точки с числовыми классами переходят в классические 0/1. Меняется только масштаб, который нужно обработать.